

```
{-# LANGUAGE OverloadedStrings #-}
import Web.Scotty

main :: IO ()
main = scotty 3000 $ do
  get "/agent" $ do
    agent <- header "User-Agent"
    maybe (raise "User-Agent header not found!") text
  agent
```

You can execute the above code in a terminal using the following command:

```
$ runghc request-header.hs
Setting phasers to stun... (port 3000) (ctrl-c to quit)
```

If you open the URL `localhost:3000/agent` in the browser, it returns the following User-Agent information on Ubuntu 14.10: *Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Ubuntu Chromium/41.0.2272.76 Chrome/41.0.2272.76 Safari/537.36*. The Curl version is returned for the same URL request as shown below:

```
$ curl localhost:3000/agent -v

* Hostname was NOT found in DNS cache
*   Trying 127.0.0.1...
* Connected to localhost (127.0.0.1) port 3000 (#0)
> GET /agent HTTP/1.1
> User-Agent: curl/7.37.1
> Host: localhost:3000
> Accept: */*
>
< HTTP/1.1 200 OK
< Transfer-Encoding: chunked
< Date: Wed, 29 Apr 2015 07:46:21 GMT
* Server Warp/3.0.12.1 is not blacklisted
< Server: Warp/3.0.12.1
< Content-Type: text/plain; charset=utf-8
<
* Connection #0 to host localhost left intact

curl/7.37.1
```

You can also return different content types (HTML, text, JSON) based on the request. For example:

```
-- content-type.hs

{-# LANGUAGE OverloadedStrings #-}

import Web.Scotty as W
import Data.Monoid
import Data.Text
```

```
import Data.Aeson

main :: IO ()
main = scotty 3000 $ do
  get "/hello" $ do
    html $ mconcat ["<h1>", "Hello, World!", "</h1>"]

  get "/hello.txt" $ do
    text "Hello, World!"

  get "/hello.json" $ do
    W.json $ object ["text" .= ("Hello, World!" :: Text)]
```

You can start the above server in a terminal as follows:

```
$ runghc content-type.hs
Setting phasers to stun... (port 3000) (ctrl-c to quit)
```

You can then open the three URLs listed above in a browser to see the different output. The respective outputs when used with Curl are shown below:

```
$ curl localhost:3000/hello
<h1>Hello, World!</h1>

$ curl localhost:3000/hello.txt
Hello, World!

$ curl localhost:3000/hello.json
{"text": "Hello, World!"}
```

You can also pass parameters in the URL when you make a request. The `param` function can be used to retrieve the parameters as indicated below:

```
-- params.hs

{-# LANGUAGE OverloadedStrings #-}

import Web.Scotty
import Data.Monoid

main :: IO ()
main = scotty 3000 $ do
  get "/user" $ do
    name <- param "name"
    html $ mconcat ["<h1>Hello ", name, "</h1>"]
```

You can start the above server using the `runghc` command:

```
$ runghc params.hs
Setting phasers to stun... (port 3000) (ctrl-c to quit)
```

You can now try the URL requests with and without